Recognising Doodles: Banana, Hockey Stick or Mosquito

UE24CS352A Machine Learning · Mini-Project Write-up · [A-18] · [Abhay TS, PES1UG24AM008] · [Avyukt Ranjan, PES1UG24AM060]

1. Problem statement

Classify 28 × 28 grayscale hand-drawn sketches from Google's Quick, Draw! collection into three classes (**banana**, **hockey stick**, **mosquito**), and compare classical machine-learning models (logistic regression and SVMs with four kernels) against a convolutional neural network (CNN). Banana and hockey stick sketches are both elongated curves, which makes that pair the main source of confusion.

2. Dataset

Three NumPy arrays (*banana.npy*, *hockey_stick.npy*, *mosquito.npy*) of uint8 28 × 28 images, with 307,936, 130,110 and 123,029 drawings respectively. We used the first **20,000 per class** (60,000 total, perfectly balanced) and did not binarise pixels. A single stratified split with seed 42 gave **48,000 train / 6,000 validation / 6,000 test** images (2,000 per class in validation and test).



Figure 1. Sample drawings (rows: banana, hockey stick, mosquito).

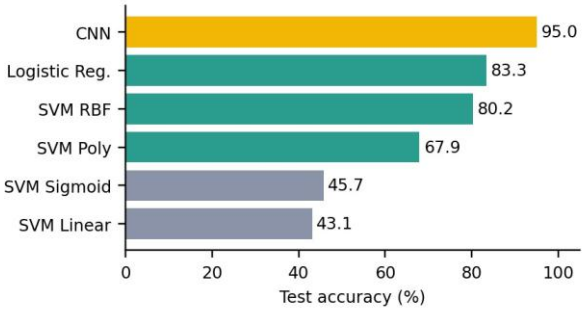


Figure 2. Test accuracy of all six models.

3. Approach

Every model is trained on the same train partition and scored once on the same held-out test partition, so results are directly comparable. Pixel scaling is fixed rather than learned, so no information leaks from the test set. Models and settings:

- **Logistic regression:** multinomial, lbfgs solver, 100 iterations, inputs scaled to [0, 1].
- **SVMs:** linear, RBF (gamma = 1.0), polynomial (degree 5, coef0 = 1.0) and sigmoid (coef0 = 1.0) kernels, each capped at 500 iterations, inputs scaled to [-1, 1].
- **CNN:** Conv(1→32, 3×3) + ReLU + MaxPool → Conv(32→64, 3×3) + ReLU + MaxPool → Flatten (3,136) → Linear 128 + ReLU → Linear 3. Adam, learning rate 0.001, batch size 64, 20 epochs, cross-entropy loss, inputs in [0, 1].

4. Implementation overview

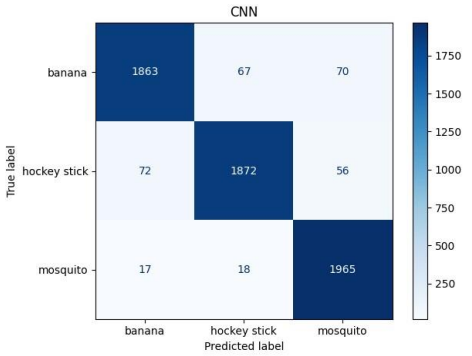
Implemented in a single Python notebook using NumPy, scikit-learn (LogisticRegression, SVC, metrics) and PyTorch (CNN), run on CPU. Steps: load and validate the arrays, balance classes, create the stratified split, scale inputs, train each model while timing training and inference, then compute accuracy, macro precision, macro recall, macro F1 and confusion matrices on the test set. The repository README explains how to set up the environment and run the notebook.

5. Results (held-out test set, 6,000 images)

Model	Accuracy	Macro-F1	Train (s)	Inference (s)
CNN	**0.9500**	**0.9499**	**679.45**	**4.480**
Logistic Regression	0.8335	0.8330	14.83	0.033
SVM RBF	0.8023	0.8034	43.23	3.759

SVM Polynomial	0.6793	0.6876	39.21	4.936
SVM Sigmoid	0.4573	0.3908	44.24	2.940
SVM Linear	0.4308	0.4352	37.93	3.279

Table 1. Test-set metrics. Rows are ordered by accuracy.



Class-level errors. The CNN confused banana as hockey stick 67 times and hockey stick as banana 72 times, versus 364 and 238 for logistic regression. Mosquito is the most distinctive shape: the CNN got 1,965 of 2,000 right (logistic regression 1,807).

Efficiency. The CNN needed about 11 minutes of CPU training; logistic regression needed 15 seconds.

Figure 3. CNN confusion matrix (rows: true class, columns: predicted).

6. Conclusions

- The CNN is the strongest model on every metric (95.0% accuracy, 0.950 macro-F1), about 11.7 points above logistic regression, because convolutions capture local stroke shapes that raw-pixel models cannot.
- Logistic regression is a strong, cheap baseline (83.4% in 15 s). RBF is the best SVM kernel (80.2%); linear and sigmoid kernels stay below 46%.
- **Limitations:** every SVM hit the 500-iteration cap ("solver terminated early") and none were tuned, so their ranking reflects this capped setup. Validation learning curves were not tracked, so overfitting was not measured directly (final CNN training loss was 0.0100). Transfer learning was not run because pretrained weights fall outside the three permitted input arrays.
- **Future work:** tune SVM hyperparameters and iteration limits, use the validation split for model selection and early stopping, and add data augmentation.